Dr. Dobb's Digest August 2009

Design for Manycore Systems

Why worry about "manycore" today?

By Herb Sutter

Herb Sutter is a bestselling author and consultant on software development topics, and a software architect at Microsoft. He can be contacted at www.gotw.ca.

Dual- and quad-core computers are obviously here to stay for mainstream desktops and notebooks. But do we really need to think about "many-core" systems if we're building a typical mainstream application right now? I find that, to many developers, "many-core" systems still feel fairly remote, and not an immediate issue to think about as they're working on their current product.

This column is about why it's time right now for most of us to think about systems with lots of cores. In short: Software is the (only) gating factor; as that gate falls, hardware parallelism is coming more and sooner than many people yet believe.

Recap: What "Everybody Knows"

Figure 1 is the canonical "free lunch is over" slide showing major mainstream microprocessor trends over the past 40 years. These numbers come from Intel's product line, but every CPU vendor from servers (e.g., Sparc) to mobile devices (e.g., ARM) shows similar curves, just shifted slightly left or right. The key point is that Moore's Law is still generously delivering transistors at the rate of twice as many per inch or per dollar every couple of years. Of course, any exponential growth curve must end, and so eventually will Moore's Law, but it seems to have yet another decade or so of life left.

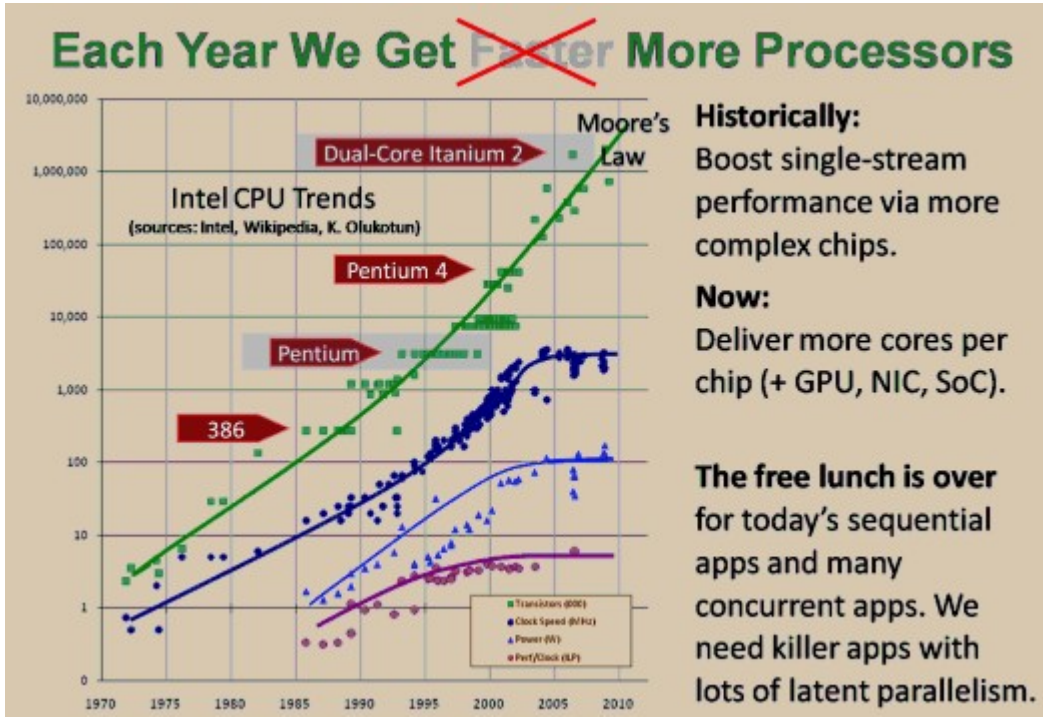

Figure 1: Canonical "free lunch is over" slide. Note Pentium vs. dual-core Itanium transistor counts.

Mainstream microprocessor designers used to be able to use their growing transistor budgets to make single-threaded code faster by making the chips more complex, such as by adding out-of-order ("OoO") execution, pipelining, branch prediction, speculation, and other techniques. Unfortunately, those techniques have now been largely mined out. But CPU designers are still reaping Moore's harvest of transistors by the boatload, at least for now. What to do with all those transistors? The main answer is to deliver more cores rather than more complex cores. Additionally, some of the extra transistor real estate can also be soaked up by bringing GPUs, networking, and/or other functionality on-chip as well, up to putting an entire "system on a chip" (aka "SoC") like the Sun UltraSPARC T2.

How Much, How Soon?

How quickly can we expect more parallelism in our chips? The nave answer would be: Twice as many cores every couple of years, just continuing on with Moore's Law. That's the baseline projection approximated in Figure 2, assuming that some of the extra transistors aren't also used for other things.

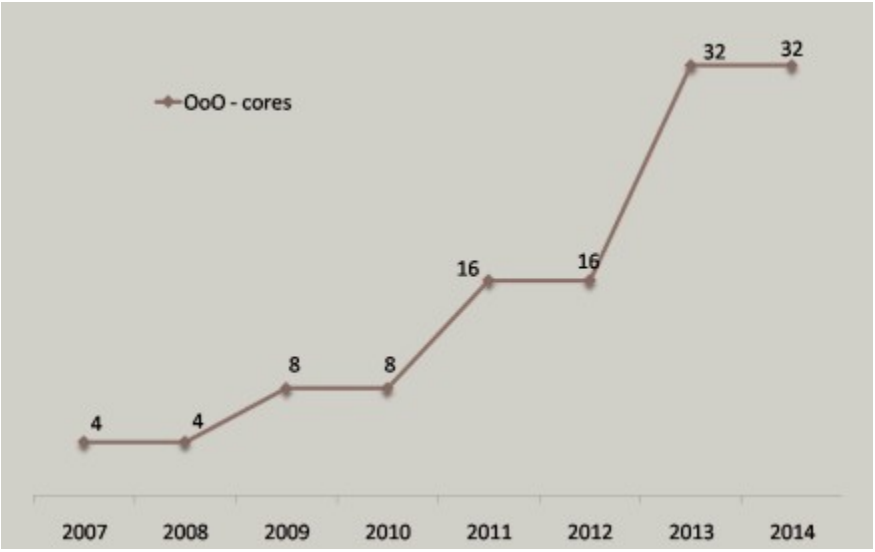

Figure 2: Simple extrapolation of "more of the same big cores" (not counting some transistors being used for other things like on-chip GPUs, or returning to smaller cores).

However, the naive answer misses several essential ingredients. To illustrate, notice one interesting fact hidden inside Figure 1. Consider the two highlighted chips and their respective transistor counts in million transistors (Mt):

- 4.5Mt: 1997 "Tillamook" Pentium P55C. This isn't the original Pentium, it's a later and pretty attractive little chip that has some nice MMX instructions for multimedia processing. Imagine running this 1997 part at today's clock speeds.
- 1,700Mt: 2006 "Montecito" Itanium 2. This chip handily jumped past the billion-transistor mark to deliver two Itanium cores on the same die. [1]

So what's the interesting fact? (Hint: 1,700 ÷ 4.5 = ???.)

In 2006, instead of shipping a dual-core Itanium part, with exactly the same transistor budget Intel could have shipped a chip that contained 100 decent Pentium-class cores with enough space left over for 16 MB of Level 3 cache. True, it's more than a matter of just etching the logic of 100 cores on one die; the chip would need other engineering work, such as in improving the memory interconnect to make the whole chip a suitably balanced part. But we can view those as being relatively 'just details' because they don't require engineering breakthroughs.

Repeat: Intel could have shipped a 100-core desktop chip with ample cache -- in 2006. So why didn't they? (Or AMD? Or Sun? Or anyone else in the mainstream market?) The short answer is the counter-question: Who would buy it? The world's popular mainstream client applications are largely single-threaded or nonscalably multithreaded, which means that existing applications create a double disincentive:

- They couldn't take advantage the extra cores, because they don't contain enough inherent parallelism to scale well.
- They wouldn't run as fast on a smaller and simpler core, compared to a bigger core that contains extra complexity to run single-threaded code faster.

Astute readers might have noticed that when I said, "why didn't Intel or Sun," I left myself open to contradiction, because Sun (in particular) did do something like that already, and Intel is doing it now. Let's find out what, and why.

Hiding Latency: Complex Cores vs. Hardware Threads

One of the major reasons today's modern CPU cores are so big and complex, to make single-threaded applications run faster, is that the complexity is u

to hide the latency of accessing glacially slow RAM -- the "memory wall."

In general, how do you hide latency? Briefly, by adding concurrency: Pipelining, out-of-order execution, and most of the other tricks used inside complex CPUs inject various forms of concurrency within the chip itself, and that lets the CPU keep the pipeline to memory full and well-utilized and hide much of the latency of waiting for RAM. (That's a very brief summary. For more, see my machine architecture talk, available on Google video. [2])

So every chip needs to have a certain amount of concurrency available inside it to hide the memory wall. In 2006, the memory wall was higher than in 1997; so naturally, 2006 cores of any variety needed to contain more total concurrency than in 1997, in whatever form, just to avoid spending most of their time waiting for memory. If we just brought the 1997 core as-is into the 2006 world, running at 2006 clock speeds, we would find that it would spend most of its time doing something fairly unmotivating: just idling, waiting for memory.

But that doesn't mean a simpler 1997-style core can't make sense today. You just have to provide enough internal hardware concurrency to hide the memory wall. The squeezing-the-toothpaste-tube metaphor applies directly: When you squeeze to make one end smaller, some other part of the tube has to get bigger. If we take away some of a modern core's concurrency-providing complexity, such as removing out-of-order execution or some or all pipeline stages, we need to provide the missing concurrency in some other way.

But how? A popular answer is: Through hardware threads. (Don't stop reading if you've been burned by hardware threads in the past. See the sidebar "Hardware Threads Are Important, But Only For Simpler Cores.")

Hardware Threads Are Important, But Only For Simpler Cores

Hardware threads have acquired a tarnished reputation. Historically, for example, Pentium hyperthreading has been a mixed blessing in practice; it made some applications run something like 20% faster by hiding some remaining memory latency not already covered in other ways, but made other applications actually run slower because of increased cache contention and other effects. (For one example, see [3].)

But that's only because hardware threads are for hiding latency, and so they're not nearly as useful on our familiar big, complex cores that already contain lots of other latency-hiding concurrency. If you've had mixed or negative results with hardware threads, you were probably just using them on complex chips where they don't matter as much.

Don't let that turn you off the idea of hardware threading. Although hardware threads are a mixed bag on complex cores where there isn't much remaining memory latency left to hide, they are absolutely essential on simpler cores that aren't hiding nearly enough memory latency in other ways, such as simpler in-order CPUs like Niagara and Larrabee. Modern GPUs take the extreme end of this design range, making each core very simple (typically not even a general-purpose core) and relying on lots of hardware threads to keep the core doing useful work even in the face of memory latency.

—HS

Toward Simpler, Threaded Cores

What are hardware threads all about? Here's the idea: Each core still has just one basic processing unit (arithmetic unit, floating-point unit, etc.) but can keep multiple threads of execution "hot" and ready to switch to quickly as others stall waiting for memory. The switching cost is just a few cycles; it's nothing remotely similar to the cost of an operating system-level context switch. For example, a core with four hardware threads can run the first thread until it encounters a memory operation that forces it to wait, and then keep doing useful work by immediately switching to the second thread and executing that until it also has to wait, and then switching to the third until it also waits, and then the fourth until it also waits -- and by then hopefully the first or second is ready to run again and the core can stay busy. For more details, see [4].

The next question is, How many hardware threads should there be per core? The answer is: As many as you need to hide the latency no longer hidden by other means. In practice, popular answers are four and eight hardware threads per core. For example, Sun's Niagara 1 and Niagara 2 processors are ba on simpler cores, and provide four and eight hardware threads per core, respectively. The UltraSPARC T2 boasts 8 cores of 8 threads each, or 64 hardw

threads, as well as other functions including networking and I/O that make it a "system on a chip." [5] Intel's new line of Larrabee chips is expected to range from 8 to 80 (eighty) x86-compatible cores, each with four or more hardware threads, for a total of 32 to 320 or more hardware threads per CPU chip. [6] [7]

Figure 3 shows a simplified view of possible CPU directions. The large cores are big, modern, complex cores with gobs of out-of-order execution, branch prediction, and so on.

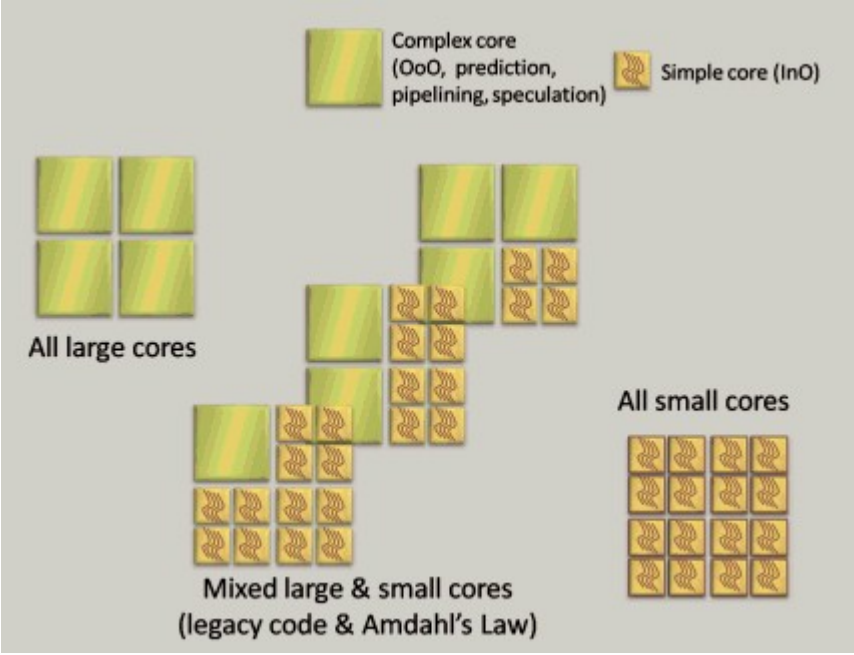

Figure 3: A few possible future directions.

The left side of Figure 3 shows one possible future: We could just use Moore's transistor generosity to ship more of the same -- complex modern cores as we're used to in the mainstream today. Following that route gives us the projection we already saw in Figure 2.

But that's only one possible future, because there's more to the story. The right side of Figure 3 illustrates how chip vendors could swing the pendulum partway back and make moderately simpler chips, along the lines that Sun's Niagara and Intel's Larrabee processors are doing.

In this simple example for illustrative purposes only, the smaller cores are simpler cores that consume just one-quarter the number of transistors, so that four times as many can fit in the same area. However, they're simpler because they're missing some of the machinery used to hide memory latency; to make up the deficit, the small cores also have to provide four hardware threads per core. If CPU vendors were to switch to this model, for example, we would see a one-time jump of 16 times the hardware concurrency -- four times the number of cores, and at the same time four times as many hardware threads per core -- on top of the Moore's Law-based growth in Figure 2.

What makes smaller cores so appealing? In short, it turns out you can design a small-core device such that:

- 4x cores = 4x FP performance: Each small, simple core can perform just as many floating-point operations per second as a big, complex core. After all, we're not changing the core execution logic (ALU, FPU, etc.); we're only changing the supporting machinery around it that hides the memory latency, to replace OoO and predictors and pipelines with some hardware threading.
- Less total power: Each small, simple core occupies one-quarter of the transistors, but uses less than one-quarter the total power.

Who wouldn't want a CPU that has four times the total floating-point processing throughput and consumes less total power? If that's possible, why not just ship it tomorrow?

You might already have noticed the fly in the ointment. The key question is: Where does the CPU get the work to assign to those multiple hardware threads? The answer is, from the same place it gets the work for multiple cores: From you. Your application has to provide the software threads or other parallel work to run on those hardware threads. If it doesn't, then the core will be idle most of the time. So this plan only works if the software is scalably parallel.

Imagine for a moment that we live in a different world, one that contains several major scalably parallel "killer" applications -- applications that a lot of mainstream consumers want to use and that run better on highly parallel hardware. If we have such scalable parallel software, then the right-hand side of Figure 3 is incredibly attractive and a boon for everyone, including for end users who get much more processing clout as well as a smaller electricity bill.

In the medium term, it's quite possible that the future will hold something in be-tween, as shown in the middle of Figure 3: heterogeneous chips that contain both large and small cores. Even these will only be viable if there are scalable parallel ap-plications, but they offer a nice migration path from today's applications. The larger cores can run today's applications at full speed, with ongoing incremental improve-ments to sequential performance, while the smaller cores can run tomorrow's applica-tions with a reenabled "free lunch" of exponential improvements to CPU-bound per-formance (until the program becomes bound by some other factor, such as memory or network I/O). The larger cores can also be useful for faster execution of any unavoidably sequential parts of new parallel applications. [8]

How Much Scalability Does Your Application Need?

So how much parallel scalability should you aim to support in the application you're working on today, assuming that it's compute-bound already or you can add killer features that are compute-bound and also amenable to parallel execution? The answer is that you want to match your application's scalability to the amount of hardware parallelism in the target hardware that will be available during your application's expected production or shelf lifetime. As shown in Figure 4, that equates to the number of hardware threads you expect to have on your end users' machines.

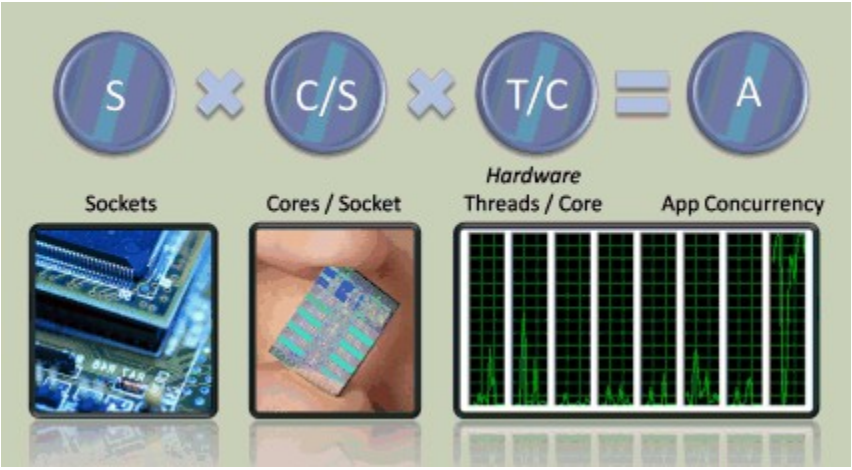

Figure 4: How much concurrency does your program need in order to exploit given hardware?

Let's say that YourCurrentApplication 1.0 will ship next year (mid-2010), and you expect that it'll be another 18 months until you ship the 2.0 release (early 2012) and probably another 18 months after that before most users will have upgraded (mid-2013). Then you'd be interested in judging what will be the likely mainstream hardware target up to mid-2013.

If we stick with "just more of the same" as in Figure 2's extrapolation, we'd expect aggressive early hardware adopters to be running 16-core machines (possibly double that if they're aggressive enough to run dual-CPU workstations with two sockets), and we'd likely expect most general mainstream users to have 4-, 8- or maybe a smattering of 16-core machines (accounting for the time for new chips to be adopted in the marketplace).

But what if the gating factor, parallel-ready software, goes away? Then CPU vendors would be free to take advantage of options like the one-time 16-fold hardware parallelism jump illustrated in Figure 3, and we get an envelope like that shown in Figure 5.

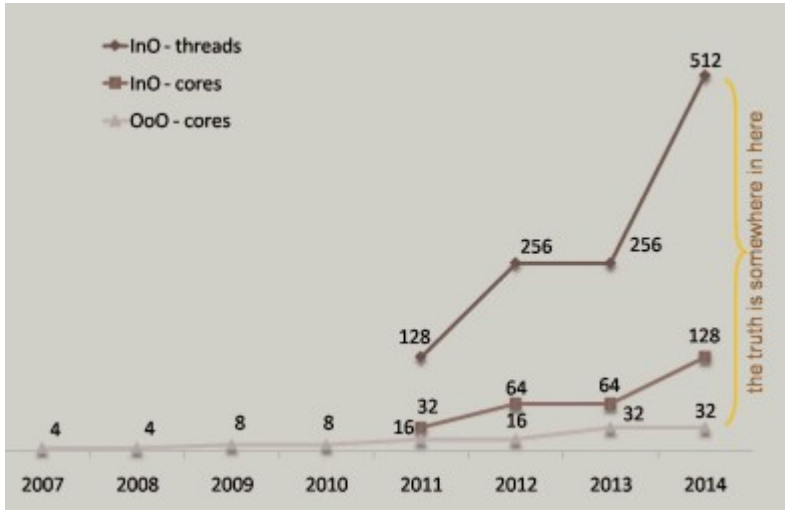

Figure 5: Extrapolation of "more of the same big cores" and "possible one-time switch to 4x smaller cores plus 4x threads per core" (not counting some transistors being used for other things like on-chip GPUs).

Now, what amount of parallelism should the application you're working on now have, if it ships next year and will be in the market for three years? And what does that answer imply for the scalability design and testing you need to be doing now, and the hardware you want to be using at least part of the time in your testing lab? (We can't buy a machine with 32-core mainstream chip yet, but we can simulate one pretty well by buying a machine with four eight-core chips, or eight quad-core chips It's no coincidence that in recent articles I've often shown performance data on a 24-core machine, which happens to be a four-socket box with six cores per socket.)

Note that I'm not predicting that we'll see 256-way hardware parallelism on a typical new Dell desktop in 2012. We're close enough to 2011 and 2012 that if chip vendors aren't already planning such a jump to simpler, hardware-threaded cores, it's not going to happen. They typically need three years or so of lead time to see, or at least anticipate, the availability of parallel software that will use the chips, so that they can design and build and ship them in their normal development cycle.

I don't believe either the bottom line or the top line is the exact truth, but as long as sufficient parallel-capable software comes along, the truth will probably be somewhere in between, especially if we have processors that offer a mix of large- and small-core chips, or that use some chip real estate to bring GPUs or other devices on-die. That's more hardware parallelism, and sooner, than most mainstream developers I've encountered expect.

Interestingly, though, we already noted two current examples: Sun's Niagara, and Intel's Larrabee, already provide double-digit parallelism in mainstream hardware via smaller cores with four or eight hardware threads each. "Manycore" chips, or perhaps more correctly "manythread" chips, are just waiting to enter the mainstream. Intel could have built a nice 100-core part in 2006. The gating factor is the software that can exploit the hardware parallelism; that is, the gating factor is you and me.

Summary

The pendulum has swung toward complex cores nearly far as it's practical to go. There's a lot of performance and power incentive to ship simpler cores. But the gating factor is software that can use them effectively; specifically, the availability of scalable parallel mainstream killer applications. The only thing I can foresee that could prevent the widespread adoption of manycore mainstream systems in the next decade would be a complete failure to find and build some key parallel killer apps, ones that large numbers of people want and that work better with lots of cores. Given our collective inventiveness, coupled with the parallel libraries and tooling now becoming available, I think such a complete failure is very unlikely.

As soon as mainstream parallel applications become available, we will see hardware parallelism both more and sooner than most people expect. Fasten your seat belts, and remember Figure 5.

References

[1] Montecito press release (Intel, July 2006) www.intel.com/pressroom/archive/releases/20060718comp.htm.

[2] H. Sutter. "Machine Architecture: Things Your Programming Language Never Told You" (Talk at NWCPP, September 2007). http://video.google.com/videoplay?docid=-4714369049736584770

[3] "Improving Performance by Disabling Hyperthreading"(Novell Cool Solutions feature, October 2004). www.novell.com/coolsolutions/feature/637.html

[4] J. Stokes. "Introduction to Multithreading, Superthreading and Hyperthreading" (Ars Technica, October 2002). http://arstechnica.com/old/content/2002/10/hyperthreading.ars

[5] UltraSPARC T2 Processor(Sun). www.sun.com/processors/UltraSPARC-T2/datasheet.pdf

[6] L. Seiler et al. "Larrabee: A Many-Core x86 Architecture for Visual Computing" (ACM Transactions on Graphics (27,3), Proceedings of ACM SIGGRAPH 2008, August 2008). http://download.intel.com/technology/architecture-silicon/Siggraph_Larrabee_paper.pdf

[7] M. Abrash. "A First Look at the Larrabee New Instructions" (Dr. Dobb's, April 2009). www.ddj.com/hpc-high-performance-computing/216402188

[8] H. Sutter. "Break Amdahl's Law!" (Dr. Dobb's Journal, February 2008). www.ddj.com/cpp/205900309.